

Ubuntu Terminal Basics

A practical first tour of the Linux terminal

Chapter 1 | Ubuntu command line essentials

What you will learn

- ▶ Why the terminal is useful for repeatable work
- ▶ Navigate folders and create, inspect, and remove files
- ▶ Redirect command output and use Tab completion
- ▶ Recognize when elevated permissions can cause damage
- ▶ Open the current folder in Ubuntu's file manager

One simple idea

The terminal is a direct way to ask Ubuntu to do a task. Learn a few small commands, then reuse them throughout the course.

1.1 Who cares about the terminal?

- ▶ It gives a consistent way to describe steps in a tutorial.
- ▶ Commands are easy to repeat, share, and adapt.
- ▶ Development tools are often launched and inspected from a shell.

You do not need to become a terminal expert today.

Think of it as a tool

A graphical interface is great for many tasks. The terminal complements it when exact, repeatable commands are useful.

1.2 The terminal: a command workspace

Hint

Open a terminal with `Ctrl+Alt+T`.
The shell reads a command, runs it, then shows the result and a new prompt.

Command shape

`command option target`

Example: `ls -l ~/Documents`

Useful command families

<code>pwd, ls</code>	Find where you are and see files
<code>cd,</code> <code>mkdir</code>	Move between folders; create folders
<code>touch,</code> <code>cat</code>	Create an empty file; read its contents
<code>grep</code>	Show lines containing a word
<code>nano</code>	Edit a text file in the terminal
<code>apt,</code> <code>sudo</code>	Install software; request admin rights
<code>alias</code>	Give a frequently used command a short name

1.2 Useful command line programs (1/2)

Table 1 – the book's command toolkit, with teaching examples

Command	Example	What to expect
<code>pwd</code>	<code>pwd</code>	Prints the full path of your current folder.
<code>mkdir</code>	<code>mkdir a_folder</code>	Creates a new folder here.
<code>cd</code>	<code>cd a_folder</code>	Moves the shell into that folder.
<code>touch</code>	<code>touch a_file.txt</code>	Creates an empty file, or updates its timestamp.
<code>cat</code>	<code>cat a_file.txt</code>	Prints the contents of a text file.
<code>rm</code>	<code>rm a_file.txt</code>	Deletes a file; <code>-r</code> also removes a directory tree.

Warning: Know your location before `rm`

Run `pwd` and `ls` first. A deleted file normally does not go to the Trash.

1.2 Useful command line programs (2/2)

Table 1 – continued

Command	Example	What to expect
<code>ls</code>	<code>ls</code>	Lists files and folders.
<code>grep</code>	<code>cat a_file.txt grep robot</code>	Keeps lines that contain robot.
<code>nano</code>	<code>nano a_file.txt</code>	Opens a simple terminal text editor.
<code>sudo</code>	<code>sudo apt update</code>	Runs a command with administrator rights.
<code>apt</code>	<code>sudo apt install git</code>	Installs an Ubuntu package (after repository setup).
<code>alias</code>	<code>alias hi="echo hello"</code>	Creates a short name for a command in this shell.

Hint

This table contains all 12 programs in the source chapter. The following slides show how to use them in practice.

How a terminal command gets work done



Example: ask for your location

Type `pwd`. Ubuntu prints the full path of the current folder.

Hint

Most commands finish and return a prompt. A running program can be stopped with

`Ctrl+C`.

Search text and reuse a command

Find matching lines with grep

```
cat notes.txt | grep robot
```

cat prints the file; | passes that output to grep, which keeps lines containing robot.

Hint

A pipe | connects one command's output to the next command's input.

Create a short alias

```
alias say_hello="echo hello"  
say_hello
```

alias gives a command a shorter name in the current shell session.

1.3 Let's use it (!?) – make a practice folder

```
# Move to your home folder, then create a workspace for this exercise
cd ~
mkdir terminal_practice
cd terminal_practice
touch empty.txt
pwd
ls
```

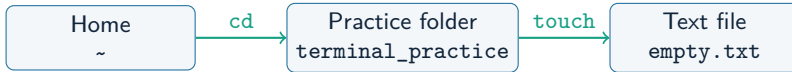
Hint

~ means your home folder. . means the folder you are in now.

What should you see?

pwd ends in /terminal_practice; ls lists empty.txt. The prompt returns after each command.

Folders are locations: read the path



Absolute path vs. current location

`/home/your-name/terminal_practice` describes the full route. After `cd terminal_practice`, the shorter name works because that folder is now the current location.

Check before changing files

Run `pwd` to confirm your location and `ls` to inspect its contents.

Create, edit, and read a text file

```
nano notes.txt
```

Type a short line, then save and exit Nano:

1. `Ctrl+X`
2. `Y` to save
3. `Enter` to confirm the filename

```
cat notes.txt  
ls
```

Hint

Nano shows its keyboard shortcuts along the bottom. The symbol `^X` means `Ctrl+X`.

Terminal copy/paste

Copy: `Ctrl+Shift+C`

Paste: `Ctrl+Shift+V`

`Ctrl+C` stops a running command.

Clean up the practice folder carefully

Warning: Pause before using `rm`

Files removed with `rm` do not go to the Trash. Check the path and target first.

```
# Go back home, then remove only the practice folder from this example
cd ~
rm -r terminal_practice
```

- ▶ Use `ls` and `pwd` to verify your location.
- ▶ Never paste an unfamiliar destructive command without understanding its target.

1.4 Bash redirections: save command output

Hint

Before writing a script for a small, common task, check whether Bash already has a simple command for it.



Replace: >

`ls > files.txt` saves the listing. Creates the file if needed; replaces old contents if it already exists.

Add: »

`echo "done" » files.txt` adds a line at the end and keeps existing contents.

Warning: Check the operator before pressing Enter

> overwrites the target file immediately; » appends to it.

1.5 Tab completion: type less, check more

Try it

1. Type `cd ~` and press Enter.
2. Type `ls` to see the available names.
3. Start a command, such as `cat result_`.
4. Press `Tab` to complete a unique match.

If more than one name matches

Press `Tab` twice to display possible matches. Add another letter or two, then press `Tab` again.

Hint

Tab completion works for commands, folders, and files. Use it often to avoid long-path typos.

1.6 Be careful with sudo

Warning: Use elevated privileges only when needed

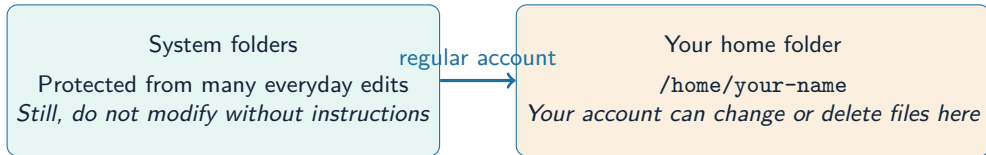
sudo runs a command with administrator privileges. A mistake can affect the whole system.

- ▶ In this course, use sudo for system-wide package installation when instructed.
- ▶ Do not add sudo just to make an error message disappear.
- ▶ Read the command and its target before entering your password.

Expected course pattern

A package installation may need `sudo apt install ...`; ordinary work in your project folder should not.

1.7 Be careful even without sudo



Warning: Your home folder is yours to change

`rm` does not need `sudo` to delete files you own. Before a change, check the target with `pwd` and `ls`.

1.8 File permissions: leave system ownership alone

Owner
Can read / write / run

Group
Access depends on rules

Others
Access depends on rules

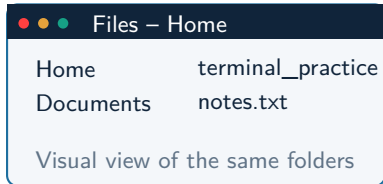
- ▶ Permissions decide who may read, change, or run a file.
- ▶ Saving a file as administrator may make it owned by the administrator account.
- ▶ Then your normal account may be unable to edit it or launch desktop tools that need it.

Warning: Beginner rule

Avoid experimenting with `chmod` (change permissions), `chown` (change ownership), or administrator-mode editors. Follow a specific, trusted instruction if a permission change is ever required.

1.9 Nautilus: browse files with a GUI

- ▶ Nautilus is Ubuntu's default file manager, similar to File Explorer or Finder.
- ▶ Open the folder you are already working in, then use the visual browser to inspect files.



Hint

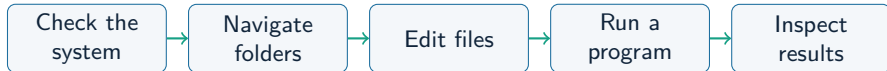
The dot `.` is a path shortcut for the current folder.

```
cd ~  
nautilus .
```

What this does

`cd ~` moves to Home; `nautilus .` opens the current folder in the visual file browser.

A repeatable Linux workflow



```
mkdir -p ~/linux_lab/src  
cd ~/linux_lab  
pwd
```

Hint

This creates a safe practice folder in your Home directory. In the following examples, `linux_lab` is your own small project.

Identify the system and get help

```
cat /etc/os-release # distribution and version
echo "$SHELL"       # login shell
command -v python3  # path to a program
python3 --help      # program options
man ls              # detailed manual; q quits
```

Read the answer

`command -v python3` prints the executable path if Python is installed. If a command is unknown, check its spelling, installation, and shell PATH.

Try this

Compare `ls -help` with `man ls`. Find the option for showing hidden files.

Paths: absolute, relative, and special names

```
cd ~/linux_lab/src # home-relative path
pwd                # absolute path, starting with /
cd ..              # move to parent folder
cd .               # stay in current folder
cd ~               # return home
```

Absolute path

Starts with /. Example:
/home/student/linux_lab/src identifies
the same place from anywhere.

Hint

. means here; .. means parent; ~ means
your Home directory.

Create, copy, move, and remove files

```
cd ~/linux_lab
touch notes.txt
cp notes.txt notes_copy.txt # duplicate a file
mv notes_copy.txt old.txt   # rename or move a file
ls
rm old.txt                  # remove only this practice file
```

Predict the listing

Before `rm`, `ls` shows `notes.txt`, `old.txt`, and `src`. After `rm`, `old.txt` is gone.

Warning: Check the exact target

`rm` acts immediately. Use `pwd` and `ls` before removal. A directory needs `rm -r`; never use that option casually.

Read and search text files

```
printf 'alpha\nbeta\nalpha again\n' > ~/linux_lab/notes.txt
cat ~/linux_lab/notes.txt
grep 'alpha' ~/linux_lab/notes.txt
wc -l ~/linux_lab/notes.txt
less ~/linux_lab/notes.txt # press q to quit
```

Expected result

grep returns two matching lines; wc -l reports three lines. less lets you scroll through a longer file.

Hint

head -n 5 file.txt shows the first five lines; tail -n 5 file.txt shows the last five.

Find files and use patterns carefully

```
cd ~/linux_lab
find . -name '*.txt'      # search this tree
ls *.txt                  # list matching names here
grep -R 'alpha' .         # search text in this tree
```

What the star means

In `ls *.txt`, Bash expands `*.txt` to matching filenames before `ls` runs. In `find`, the quoted pattern is passed to the program.

Warning: Inspect a pattern before deletion

First run `ls *.txt`. Then, if you really mean every match, use a command that changes files. Patterns can match more than you expect.

Quote paths and arguments with spaces

```
mkdir -p ~/linux_lab/'My Files'  
touch ~/linux_lab/'My Files'/report.txt  
cd ~/linux_lab/'My Files'  
pwd
```

Why quotes matter

Without quotes, Bash treats `My` and `Files` as separate arguments. The single quotes keep them together as one path.

Hint

Tab completion can insert the needed escape characters for a filename with spaces.

Install Ubuntu packages with apt

```
sudo apt update      # refresh package lists
apt search git       # search available packages
apt show git         # inspect version and description
sudo apt install git # install the selected package
```

Understand the sequence

update refreshes the package index; install changes the system. sudo is needed for system package changes, so read the package name first.

Hint

apt list -installed git checks whether git is installed. Package availability depends on configured repositories.

Environment variables: settings in a shell

```
echo "$HOME"           # home directory
echo "$PATH"           # program search path
export PROJECT_DIR="$HOME/linux_lab" # set a variable
echo "$PROJECT_DIR"
printenv | grep PROJECT_DIR
```

What changed?

export makes PROJECT_DIR available to programs started by this shell. A separate terminal does not inherit changes made afterward in this one.

Hint

Quotes around "\$PROJECT_DIR" keep a value containing spaces together as one argument.

source and .bashrc: current vs. new shells

```
cd ~/linux_lab
printf 'export DEMO_MODE=ready\n' > demo_env.sh
source ./demo_env.sh
echo "$DEMO_MODE" # expected: ready
```

Why source is different

It runs the script in the current shell, so the variable remains available there. Running the script as a separate process would not modify the parent shell.

Hint

Bash reads ~/.bashrc when an interactive shell opens. Put only intentional, reviewed startup commands in it.

Permissions: read, write, and execute

```
cd ~/linux_lab
printf '#!/usr/bin/env bash\nnecho ready\n' > check.sh
ls -l check.sh
chmod u+x check.sh
./check.sh           # expected: ready
```

Read `chmod u+x`

`u` means the file's owner; `+x` adds execute permission for that owner. `./check.sh` runs the script in the current folder.

Warning: Make only the needed change

Avoid `chmod 777` and arbitrary `chown` commands. Inspect `ls -l` and the error message first.

Processes: start, observe, and stop

```
sleep 30          # runs in the foreground; Ctrl+C stops it
sleep 30 &        # runs in the background
jobs              # show shell jobs
kill %1           # stop job 1 from this shell
```

Foreground vs. background

A foreground command occupies the terminal until it ends or you stop it. A trailing `&` returns the prompt while the command runs.

Hint

`ps -ef | grep sleep` can help find a process. A new terminal has its own job list.

Pipes and redirections: route program output

```
cd ~/linux_lab
ls | sort           # send output into another command
ls > files.txt      # create or replace a file
ls >> files.txt     # append to the file
ls | tee listing.txt # show and save at the same time
```

Read from left to right

The pipe `|` connects two commands. `sort` receives the names printed by `ls`.

Warning: `>` replaces data

It does not ask before overwriting an existing file. `>>` adds lines instead.

Errors and logs: capture evidence

```
cd ~/linux_lab
ls missing.txt 2> errors.log # save error output
cat errors.log
tail -n 10 errors.log      # last ten lines
```

Two output streams

Normal output goes to standard output; error messages go to standard error. `2>` redirects errors to a file, while `>` redirects normal output.

Hint

`tail -f app.log` follows a growing log. Press `Ctrl+C` to stop following it.

Git: track changes in a project folder

```
cd ~/linux_lab
git init
printf '# Linux lab\n' > README.md
git status
git add README.md
git status
```

What to observe

The first `git status` shows an untracked file. After `git add`, it shows the file staged. No commit is made in this example.

Hint

When using someone else's repository, read its README and check the branch before editing or building.

Python projects: isolate dependencies

```
cd ~/linux_lab
python3 --version
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
deactivate
```

What activation changes

The active shell uses the project's Python and pip. deactivate returns to the prior environment. The folder `.venv/` belongs to this project.

Hint

If `venv` is unavailable, inspect the Ubuntu package `python3-venv`. Install packages only after checking the project's instructions.

Network basics: inspect, then test

```
hostname          # this machine's name
hostname -I       # local IP addresses
ip -brief address # interfaces and addresses
ping -c 3 127.0.0.1 # test the local network stack
```

What these commands tell you

An address identifies a network interface. The loopback address 127.0.0.1 reaches this computer only; it does not test another machine.

Hint

When two machines must communicate, record both addresses and verify each can reach the other before debugging the application.

Device files: inspect access before changing it

```
ls /dev/ttyUSB*      # list matching serial devices, if any
ls -l /dev/ttyUSB0   # owner, group, and permissions
groups                # your account's groups
```

Why device files matter

USB serial adapters and other hardware may appear under `/dev`. A program must have permission to open the relevant device file.

Warning: Use the hardware instructions

The correct group or `udev` rule depends on the device. Avoid broad permission changes; inspect the device first.

Troubleshooting: start from the symptom

Symptom	First useful command
Command not found	<code>command -v name</code> ; check installation and PATH.
File not found	<code>pwd</code> , <code>ls</code> , then <code>find . -name 'file'</code> .
Permission denied	<code>ls -l file</code> ; inspect owner and group.
Program seems stuck	<code>ps -ef</code> ; identify the process, then use <code>Ctrl+C</code> for a foreground program.
Unexpected output	Capture error text with <code>2> errors.log</code> ; read it with <code>cat</code> or <code>less</code> .

Hint

Record the exact command, folder, and first error message. That evidence usually points to the next check.

Mini lab: one small project from start to result

```
mkdir -p ~/linux_lab/src
cd ~/linux_lab
printf 'print("hello from Linux")\n' > src/hello.py
python3 src/hello.py | tee run.log
grep 'hello' run.log
ls -lah src run.log
```

Expected result

Python prints hello from Linux; tee saves the same line in run.log; grep finds that line. ls confirms the script and log exist.

Explain each step

Which command created a folder? Which wrote a file? Which ran a program? Which showed and saved the result?

Linux readiness checklist

1. I can check my Ubuntu version and get command help: `cat /etc/os-release`, `-help`.
2. I can navigate and inspect files: `pwd`, `cd`, `ls`, `find`.
3. I can edit, search, and route text: `nano`, `grep`, `|`, `>`, `tee`.
4. I can install an Ubuntu package with `apt` when needed.
5. I understand shell variables, `source`, and `~/.bashrc`.
6. I can inspect permissions, a process, a log, and a device file.
7. I can create and run the mini lab script and explain its output.

Practice check: can you explain each command?

Read first, then predict what happens

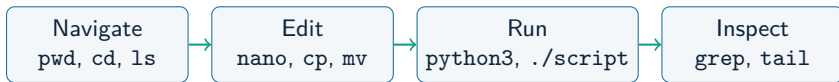
```
cd ~ → mkdir demo → cd demo → touch note.txt → ls
```

1. Which command moves you to the home folder?
2. What does `touch` do in this example?
3. What should `ls` display?
4. Which operator appends output: `>` or `»`?
5. Why should you check the target before running `rm -r`?

Expected result

`ls` should show `note.txt`. The `»` operator appends. `rm -r` can permanently remove its target, so inspect the path and name first.

Quick recap



- ▶ Confirm paths and file targets before editing, redirecting, or deleting.
- ▶ Use apt for system packages and a project environment for local Python dependencies.
- ▶ Remember that each terminal has its own variables and running jobs; use `Ctrl+C` to stop a foreground command.
- ▶ Diagnose failures through error text, logs, paths, processes, and permissions.

Sources and next reading

Chapter basis

Sections 1.1–1.9 and the twelve-command table are adapted from the supplied *Ubuntu Terminal Basics* chapter.

Built-in references

Try `man bash`, `man ls`, `man chmod`, and a program's `-help` option.

The file and script exercises use the `~/linux_lab` practice folder; package management is shown separately.

Hint

Read each command and its expected effect before running it, especially commands that install packages or remove files.